

2 de octubre de 2026

Ejercicio 1

Consigna

La empresa de logística **LogiTech** necesita renovar su módulo de gestión de paquetes. Se te solicita diseñar e implementar una solución en Java aplicando los principios de POO y organizar el código en paquetes.

1. Módulo Git

- Inicializa/utiliza un repositorio Git.
- Crea una rama de desarrollo llamada `feature/sistema-envios`.
- Realiza commits frecuentes con mensajes claros (ejemplo: `feat: agrega interfaz Enviable y clase Paquete`).
- Al finalizar, fusiona la rama en `main`.

2. Paquete modelo

1. Interfaz **Enviable**:

- Método `double calcularCostoEnvio()`
- Método `boolean esAptoParaEnvioAereo()`

2. Clase **Abstracta Paquete** (Implementa **Enviable**):

- Atributos privados: `codigoTrack` (String), `pesoKg` (double), `destino` (String).
- Constructor que inicialice todos los atributos, asegurando mediante validaciones que el peso sea estrictamente mayor a 0 y que el código no sea nulo ni esté vacío (en caso contrario, lanzar `IllegalArgumentException`).
- Getters y Setters correspondientes con validaciones.
- **Sobrecarga de Métodos (Polimorfismo):**
 - `public void actualizarDestino(String nuevoDestino):` cambia el destino del paquete.
 - `public void actualizarDestino(String nuevoDestino, boolean express):` cambia el destino y, si `express` es verdadero, le concatena la leyenda " `[PRIORITARIO]`" al destino.
- Método abstracto: `public abstract String obtenerDetalle();`

3. Clase **PaqueteEstandar** (Hereda de **Paquete**):

- Atributo privado adicional: `diasEstimados` (int).
- Constructor que invoque con `super(...)` a la superclase e inicialice `diasEstimados`.

- **Implementación de métodos:**

- `calcularCostoEnvio()`: El costo base es \$1000 * pesoKg.
- `esAptoParaEnvioAereo()`: Retorna true únicamente si pesoKg <= 15.0.
- `obtenerDetalle()`: Retorna un resumen con el código, destino, peso y días estimados.

4. Clase **PaqueteFragil** (Hereda de Paquete):

- Atributo privado adicional: `nivelProteccion` (String: "Baja", "Media", "Alta").
- Constructor con validación del nivel de protección.
- **Implementación de métodos:**
 - `calcularCostoEnvio()`: El costo base es \$1000 * pesoKg más un recargo del 30% si la protección es "Alta" o 15% si es "Media".
 - `esAptoParaEnvioAereo()`: Un paquete frágil **nunca** es apto para envío aéreo (retorna false).
 - `obtenerDetalle()`: Retorna un resumen con el código, destino, peso y nivel de protección.

3. Paquete app

1. Clase **CentroLogistico**:

- Atributo privado: `ArrayList<Envable> inventario`.
- Método `public void registrarPaquete(Envable e)`
- Método `public void mostrarReporteEnvios()`: Recorre el inventario de forma polimórfica imprimiendo el detalle del paquete y su costo total de envío.
- Método `public double calcularRecaudacionTotal()`: Suma y retorna el costo total de envío de todos los paquetes registrados.

2. Clase **Main**:

- Método `main` que instancie un `CentroLogistico`, agregue al menos 2 paquetes estándar y 2 frágiles, invoque la sobrecarga de `actualizarDestino`, pruebe la captura de excepciones con un valor inválido y muestre el reporte final.